



TP2 - ÉNONCE DE PLANIFICATION INITIALE

Membres :

Zebib, Youssef
Hamelin, Charles
Francisque, Sunny
Abbas, Hadi
Mentor, Marie Françoise Esther E

Professeur : Jacques Berger
Été 2026

Partie A - CHARTE DE PROJET

Mission du projet

L'équipe ServiceFacile existe pour offrir à la communauté une plateforme simple où n'importe qui peut publier un service dont il a besoin ou qu'il souhaite offrir, et où d'autres membres peuvent y répondre. La contribution unique de l'équipe est de réduire la friction entre les gens qui cherchent de l'aide pour une tâche ponctuelle et ceux qui sont prêts à la réaliser, sans dépendre de grandes plateformes commerciales.

Client

Le client de l'équipe est la communauté d'utilisateurs visée par la plateforme ServiceFacile, soit les personnes qui souhaitent publier ou trouver des services. Le professeur M. Jacques Berger agit comme représentant du Product Owner pour valider la portée et les livrables.

Objectifs SMART

1. Livrer une version fonctionnelle des 10 fonctionnalités essentielles d'ici la fin de la session, en respectant les priorités établies au TP1.
2. Maintenir un carnet GitLab à jour à chaque fin de sprint, avec un taux de complétion d'au moins 80 % des issues essentielles planifiées pour chaque itération.
3. Réaliser au moins une revue de code par les pairs pour chaque fonctionnalité essentielle avant son passage à l'état "Terminé".

Portée du projet

La portée détaillée a été établie dans le TP1 (TP1-Portée-ServiceFacile.xlsx, disponible sur le GitLab du projet). Le projet couvre 25 exigences réparties en 10 essentielles, 8 importantes et 7 souhaitables, couvrant la création de compte, la gestion de profil, la publication et la consultation de services, le système de candidatures, la messagerie, les avis, et les tableaux de bord utilisateur et administrateur. Le système de paiement en ligne intégré, la géolocalisation automatique et les notifications en temps réel ne font pas partie de la portée.

Matrice des leviers

Levier	Souple	Moyen	Rigide
Portée			X
Échéancier		X	
Budget	X		

La portée est rigide puisqu'elle a été validée dans le TP1 selon les attentes du cours. L'échéancier est moyennement flexible, les dates de remise sont fixes mais l'ordre interne des tâches peut être réorganisé. Le budget en heures-personne est le levier le plus souple, l'équipe peut ajuster l'effort investi par chacun sans impact direct sur la note si la portée essentielle est respectée.

Résumé des coûts anticipés

Au tarif de 110 \$/h et avec 5 membres d'équipe, l'effort total estimé pour les 25 issues, basé sur les USP du TP1, est d'environ 320 heures-personne, soit environ 35 200 \$ pour l'ensemble du projet.

Critères de succès

Du point de vue du client, le succès se mesure par la livraison de tous les livrables dans les délais prévus, la cohérence entre la portée du TP1 et le travail réalisé, la qualité du code et de la documentation, et la capacité de l'équipe à démontrer une gestion de projet rigoureuse.

Liste des risques

Risque	Mitigation	Contingence
Un membre de l'équipe ne livre pas ses tâches dans les délais	Suivi hebdomadaire sur Discord avec point d'avancement de chacun	Redistribution des tâches restantes entre les autres membres, documentée dans le carnet d'obstacles
Sous-estimation de la complexité d'une fonctionnalité (ex. messagerie, commandes)	Décomposer les grosses issues en sous-tâches plus petites avant de commencer	Réduire la portée au strict nécessaire pour respecter l'échéancier
Conflits de fusion fréquents sur le code partagé	Travailler sur des branches séparées et faire des commits fréquents	Désigner un responsable de la gestion de configuration pour résoudre les conflits
Manque de disponibilité d'un membre (examens, travail, maladie)	Prioriser les tâches critiques tôt dans le sprint	Le coordonnateur réajuste l'échéancier et redistribue la charge
Mauvaise compréhension d'une exigence menant à du travail à refaire	Revue des issues détaillées en équipe avant de développer	Refaire la partie concernée en priorité au sprint suivant

Partie B - Charte d'équipe

1. Nom d'équipe et liste des membres

Nom d'équipe : ServiceFacile

Membre	Initiales
Abbas, Hadi	HA
Francisque, Sunny	SF
Hamelin, Charles	CH
Mentor, Marie Françoise Esther E	MM
Zebib, Youssef	YZ

2. Les différentes contraintes

- L'équipe doit respecter les dates de remise et de présentations
- Chaque membre dispose d'environ 3 à 5 heure par semaine
- L'équipe est responsable de la priorisation interne des tâches, de la répartition de l'effort, des choix technologiques et de la gestion des conflits internes.
- La décision finale sur la portée (quelles issues sont attribuées à l'équipe) demeure la responsabilité du PO.

3. Rôle et responsabilités

Rôles	Responsabilités	Assigné à
Coordonnateur	Planifie les rencontres, assure le suivi de l'avancement, fait le lien avec le PO et la professeure, consolide les livrables avant dépôt	MM
Développeur(s)	Implémente les fonctionnalités assignées, écrit les tests unitaires, documente le code	HA, SF, CH, MM et YZ
Responsable AQ	S'assure du respect de la Définition de Terminé, organise les revues de code par les pairs, vérifie la couverture de tests	CH

Responsable gestion de configuration	Gère les branches Git, les fusions (merge), les conventions de nommage, l'intégrité du dépôt GitLab	CH
Point de contact technique	Communique avec les autres équipes dépendantes du même projet pour résoudre les conflits d'intégration ou de dépendances	YZ
PO (externe à l'équipe)	Valide la portée, priorise le carnet de produit, répond aux questions de clarification	Jacques Berger (pour le cours)

4. Normes de fonctionnement de l'équipe

- Chaque membre devrait respecter les délais imposés en réunion d'équipe.
- Si un membre anticipe un retard, il avise le reste de l'équipe suffisamment en avance.
- Documentation du code, le code livré doit être suffisamment commenté pour être compris par un autre membre de l'équipe.
- Qualité des commit, les messages de commit doivent être clairs et en français.
- En cas de désaccord, on l'exprime avec respect.
- Revue de code : toute "merge request" doit être approuvée par au moins un autre membre de l'équipe avant d'être fusionnée.

5. Processus de développement

Pour ce projet, nous utilisons une approche Kanban à l'aide des labels GitLab et du GitLab Issue Board afin de gérer et suivre l'avancement des tâches.

Le travail est organisé en plusieurs étapes qui va être représenté par des colonnes du Kanban :

- **Pas commencé** : les tâches qui n'ont pas encore été prises en charge
- **Prêt à développer** : les tâches prêtes à être réalisées
- **En développement** : les tâches en cours de réalisation sur une branche
- **En validation** : les tâches en cours de revue de code ou de tests
- **En vérification** : les tâches vérifiées et presque terminées
- **Terminé** : les tâches complétées selon la définition de terminé

6. Outils et leurs usages

Catégorie	Outil	Usage
Gestion de projet	GitLab	Suivi des issues, branches de code, fusions (merge requests), wiki

		(dictionnaire de données), tableau Kanban
Gestion de projet	Microsoft Excel	Suivi des activités, estimations d'effort, calculs de budget (TP2-D)
Communication	Discord	Communication quotidienne, partage rapide d'information, entraide technique et appels vocaux à distance
Documentation	Microsoft Word	Rédaction des livrables textuels (chartes, rapports de gestion de projet)
Développement	VS Code (ou IDE choisi)	Éditeur de code principal utilisé par tous les membres pour le développement de l'application
Développement	Framework frontend (À décider)	Développement de l'interface utilisateur de la plateforme ServiceFacile
Développement	Backend / Langage (À décider)	Implémentation de la logique d'affaires et des APIs de l'application
Base de données	SGBD (À décider)	Stockage et gestion des données (utilisateurs, services, commandes, etc.)
Tests	Outil de tests (À décider)	Exécution des tests unitaires et d'intégration pour valider les fonctionnalités développées

Le choix du stack technique (frontend, backend, base de données, outils de tests) sera finalisé en début de réalisation. Les estimations d'effort du livrable D restent valides peu importe l'outil choisi, puisqu'elles reposent sur la complexité des fonctionnalités plutôt que sur une technologie précise.

7. Logistique de coordination

- Rencontre principale : une fois par semaine en présentiel, après le cours magistral ou la séance de laboratoire
- Rencontre secondaire : sur Discord (appel vocal), en cas de question ou autre
- Communication continue : canal Discord dédié à l'équipe pour les questions rapides, partage de liens et mises à jour informelle.
-

8. Modes décisionnels

Mode principale : Consensus

Pour les décisions courantes (priorisation des tâches, choix techniques mineurs), l'équipe vise un accord où la majorité est favorable et personne ne s'y oppose fermement.

Mode secondaire : Autocratie par le coordonnateur.

Si un consensus n'est pas atteint dans un délai raisonnable, le coordonnateur tranche pour éviter de bloquer l'avancement, après avoir entendu tous les points de vue.

9. Comportement attendus

- Organiser et assister à des réunions d'équipe.
- Respecter les délais pris en réunion.
- Communiquer proactivement.
- Tester pour s'assurer de la validité avant un merge.

10. Comportement non tolérés

- Absence répétée sans préavis lors de la planification des sprints
- Ne pas communiquer une absence ou un retard
- Surcharger un membre de l'équipe
- Manque de respect et autres propos
- Modifier un travail d'un autre sans discussion préalable

11. Résolutions de conflit

En cas de désaccord entre deux membres, la première étape est une discussion directe entre les personnes concernées. Si le désaccord persiste, le coordonnateur intervient comme médiateur lors de la prochaine rencontre d'équipe.

Si un membre ne livre pas ses engagements, le coordonnateur en discute d'abord avec lui pour comprendre la situation. Si elle persiste sans amélioration après un avertissement documenté dans le carnet d'obstacles, l'équipe se réunit pour redistribuer les tâches restantes et documente la situation de façon factuelle, soit les dates, les tâches non complétées et les communications échangées. Cette documentation permet à l'équipe d'expliquer clairement la situation au professeur au besoin.

Partie C - « Définition de Terminé »

Une fonctionnalité est considérée comme terminée lorsqu'elle répond aux exigences du projet et qu'elle est prête à être mise en production.

1. Assurance qualité logicielle

- La fonctionnalité répond aux besoins et aux critères d'acceptation définis dans l'issue.
- Le code a été révisé par au moins un autre membre de l'équipe.
- Les tests de la fonctionnalité ont été réalisés et passent avec succès dans le pipeline CI/CD.
- Les erreurs identifiées ont été corrigées.
- Le code respecte le style définis par l'équipe.
- Le code respecte les normes de développement établies par l'équipe.
- La fonctionnalité fonctionne correctement avec les autres composants du logiciel.
- Le code compile sans avertissement, ni erreur.
- Le code utilise les fonctionnalités récentes du langage et évite l'utilisation de fonctionnalités deprecated.
- Chaque fonctionnalité est développée sur une branche individuelle.
- L'issue associée à la fonctionnalité a été fermée.
- La fonctionnalité a été présentée à l'équipe.
- Les tâches associées sont marquées comme terminées dans le gestionnaire de projet (GitLab Issue Board de type Kanban).
- Chaque fonctionnalité est associée à un ou plusieurs commits et est documentée dans le changelog

2. Assurance qualité de la documentation

- La documentation liée à la fonctionnalité a été mise à jour.
- La documentation de l'API est constamment à jour.
- Les documents ont été relus par un membre de l'équipe afin de valider la qualité.
- Les livrables respectent les exigences du cours.